# The Temporal Gap in Multi-Agent Memory: An Impossibility Result, a Causal Experiment, and a Practical Resolution

Anonymous

**Abstract**

When multiple AI agents share a memory store and one agent corrects a fact written by another, the system must decide what happens to the old value. Overwrite-style stores silently replace it, destroying temporal history. We prove that no wrapper around an overwrite-style store can support as-of queries without $\Omega(k)$ log replay. We then show this temporal gap has a direct causal impact: downstream agents receiving stale context produce 40% more errors than those receiving correct context (McNemar $p < 0.0001$, $n$=100). On a production deployment (26 days, 13 agent sources, 1,768 facts), cross-agent contradictions cause a **73-percentage-point downstream accuracy loss** (9% vs. 82%, $n$=11), while self-corrections show negligible impact (0.7pp, $n$=300). A bi-temporal store with supersession chains achieves 100% temporal accuracy on 50 real correction events, maintains flat $O(\log n)$ as-of latency (0.17–0.25 ms across $k$=1 to 500), and prevents 52–61% of potential contradictions. We release all code, data, and evaluation scripts.

## 1 Introduction

AI agents accumulate persistent memory across sessions. When a fact changes—an API key is revoked, a configuration value is updated, a tool is deprecated—the memory system must handle the transition. Two approaches exist:

1. **Overwrite**: Replace the old value in-place. The old value is irrecoverable. Systems: mem0 [1], LangMem [4].
2. **Bi-temporal supersession**: Mark the old value as superseded with explicit validity boundaries. Both values remain queryable. Systems: MemStrata [8], MemTether [5].

The overwrite approach is simpler but creates a *temporal gap*: the system can answer "what is true now?" but not "what was true at time $t$?" This gap has measurable consequences. When an agent retrieves a stale (outdated) fact and passes it to a downstream LLM, the LLM produces incorrect answers at a dramatically higher rate.

We present four contributions:

1. **Impossibility theorem** (Section 3): No wrapper around an overwrite-style store can support as-of queries without $\Omega(k)$ replay. Temporal correctness requires storage-layer support.
2. **Causal experiment** (Section 5.3): Stale context causes a 39.0pp downstream accuracy loss ($p < 0.0001$, $n$=100), establishing a causal (not merely correlational) relationship.
3. **The cross-agent asymmetry** (Section 5.4): Cross-agent contradictions cause a 73pp loss ($n$=11), while self-corrections cause only 0.7pp ($n$=300). The temporal gap is specifically a *cross-agent* problem.
4. **Practical resolution** (Section **??**): Bi-temporal supersession achieves 100% temporal accuracy, flat $O(\log n)$ latency, and prevents 52–61% of potential contradictions.

# 2 Background

## 2.1 Agent Memory Architectures

mem0 [1] provides hosted memory with LLM-based fact extraction. When `update()` is called, the old embedding is replaced in the vector store. Zep [2] offers a temporal knowledge graph. Engram [3] implements bi-temporal storage. MemTether [5] is a local-first system where 13 AI agent clients on one machine share a single SQLite database via file-level pointers, with bi-temporal supersession chains and hash-chained audit.

## 2.2 The Temporal Update Problem

When agent $A$ writes "API key $X$ is available" on day 1, and agent $B$ writes "API key $X$ returned 403" on day 5, the memory store contains two contradictory facts. If the system overwrites, agent $A$'s fact is lost. If the system retains both, a downstream agent querying "Can I use $X$?" receives contradictory evidence.

# 3 Impossibility of Wrapper-Based As-Of Queries

**Definition 1** (Overwrite-Style Store)**.** *A store $M$ that maintains a single current value per key.* `update(key, value)` *destroys the previous value.*

**Definition 2** (As-Of Query)**.** *At time $t$, return the value that was current at $t$, reflecting all updates before $t$ and none after.*

**Definition 3** (Wrapper)**.** *A layer $W$ between the agent and store $M$. $W$ intercepts* `read()` *and* `write()` *calls.*

**Theorem 1** (Wrapper As-Of Impossibility)**.** *For any overwrite-style store $M$ and wrapper $W$:*

1. *$W$ cannot answer as-of queries using $M$'s query interface alone.*
2. *$W$ must maintain auxiliary log $A$; answering as-of at time $t$ requires $\Omega(k)$ replay of $A$, where $k =$ number of updates.*
3. *If $A$ is itself bi-temporal, $W$ has replaced $M$, not extended it.*

*Proof.* (1) $M$ stores only the latest value. Historical values are not accessible. (2) $W$ must scan $A$ for the last entry $(x, v, t')$ where $t' \leq t$; worst case examines all $k$ entries. (3) If $A$ supports $O(\log n)$ as-of via indexed validity intervals, $A$ is a bi-temporal store. $W$ is redundant. □

**Corollary 1.** *Temporal correctness requires bi-temporal storage at the storage layer. It cannot be added post-hoc.*

# 4 System Design

We implement bi-temporal supersession in MemTether. Each fact is a tuple $(uid, content, status, vf, vt, ra, ia, sb)$ where $vf$ is valid-from, $vt$ is valid-to, $ra$ is recorded-at, $ia$ is invalidated-at, and $sb$ is superseded-by.

**Theorem 2** (Temporal Gap Freedom)**.** *For any supersession chain and any time $t$, the as-of query returns exactly one record.*

**Theorem 3** (No Overlap)**.** $|asOf(C, t)| \leq 1$ *for all chains and times.*

**Theorem 4** (Cycle Freedom)**.** *The supersession graph is acyclic.*

**Theorem 5** (Concurrency Safety)**.** *Concurrent* `correct()` *calls cannot fork the chain.*

Proofs rely on half-open interval continuity (Theorems 1–2), cycle detection (Theorem 3), and conditional SQL updates (Theorem 4). Full proofs in [5].

# 5 Evaluation

## 5.1 Temporal Accuracy

We construct 50 temporal accuracy questions from real production corrections (348 total supersession events over 26 days). Each question tests whether the system can retrieve both the old value ("What was true before?") and the new value ("What is true now?").

Table 1: Temporal accuracy: bi-temporal vs. overwrite.

| System | Past (old) | Now (new) | Temporal Acc. |
|---|---|---|---|
| Bi-temporal (ours) | 50/50 | 50/50 | **100%** |
| Overwrite (mem0) | 7/50 | 50/50 | 14% |
| Overwrite + log patch | 50/50 | 50/50 | 100% |

The overwrite + log patch achieves 100% accuracy but requires $O(k)$ replay for each as-of query, making it impractical at scale.

## 5.2 As-of Latency Scaling

Table 2: As-of query latency ($k$ corrections to same fact).

| $k$ | Ours (ms) | Wrapper replay (ms) |
|---|---|---|
| 1 | 0.167 | 25.5 |
| 100 | 0.212 | 26.1 |
| 500 | 0.245 | 27.0 |

Our bi-temporal store maintains flat $O(\log n)$ latency across 500× data scaling (1.47× growth).

## 5.3 Causal Experiment: Stale Context Harms Downstream Tasks

**Design.** Sample 100 supersession events. For each, construct two contexts (correct = new fact, stale = old fact). Ask GLM-5.3-flash (temperature=0): "What is the current status of [entity]?" Grade against the gold (new) fact.

Table 3: Downstream accuracy ($n$=100).

| Context | Correct | Wrong | Accuracy |
|---|---|---|---|
| Correct (new fact) | 99 | 1 | 99.0% |
| Stale (old fact) | 60 | 40 | 60.0% |
| McNemar $p < 0.0001$ | | | 39.0pp |

**Finding 1** (Stale Context Causes 39pp Loss)**.** *Receiving stale context causes a 39.0pp drop in downstream accuracy ($p < 0.0001$). This establishes a causal relationship: the temporal gap directly causes task failures.*

## 5.4 The Cross-Agent Asymmetry

We separate supersession events into two types and measure downstream impact independently.

**Cross-agent contradictions ($n$=11).** Two different agents wrote contradictory facts. Results: correct context 82%, stale context 9%.

**Self-corrections ($n$=300).** One agent corrects its own earlier statement. Results: correct context 100%, stale context 99.3%.

Table 4: Downstream accuracy by conflict type.

| Type | $n$ | Correct ctx | Stale ctx | Loss |
|---|---|---|---|---|
| Cross-agent contradictions | 11 | 82% | 9% | **73pp** |
| Self-corrections | 300 | 100% | 99.3% | 0.7pp |

**Finding 2** (73pp Cross-Agent Loss)**.** *Cross-agent contradictions cause a 73pp downstream accuracy loss. Self-corrections have negligible impact. The temporal gap is specifically a* cross-agent *problem: agents correcting their own errors maintain consistency, but contradictions between agents create irrecoverable confusion.*

### 5.5 Contradiction Detection on ConflictBank

We evaluate our detection method on ConflictBank [9], a large-scale benchmark with 7.4M claim-evidence pairs. We sample the temporal conflict subset (1,000 positive + 1,000 negative pairs).

Table 5: Temporal conflict detection on ConflictBank ($n$=2,000).

| Method | Precision | Recall | $F_1$ | Latency |
|---|---|---|---|---|
| Temporal-numeric (ours) | 1.000 | 0.814 | **0.897** | 0.133 ms |
| GLM-5.3-flash | 1.000 | 0.060 | 0.113 | 1.56 s |

Our method achieves perfect precision (zero false positives) with 11,704× speedup over LLM detection.

## 6 Discussion

### 6.1 The Temporal Gap Is Architectural

Theorem 1 proves that the temporal gap is not an engineering oversight but an architectural consequence. No wrapper can bridge the gap without becoming a bi-temporal store itself.

### 6.2 The Cross-Agent Asymmetry Is the Key Insight

The 73pp vs. 0.7pp asymmetry provides a precise diagnosis: the problem is not memory staleness in general, but specifically *disagreement between agents*. This has a direct implication: memory systems serving multiple agents should prioritize cross-agent contradiction detection over generic temporal consistency checking.

### 6.3 Limitations

- $n$=11 for cross-agent contradictions (small sample).
- Single production deployment; results may vary across systems.
- Single LLM judge (GLM-5.3-flash); different models may exhibit different sensitivity to stale context.
- Detection method is specific to temporal conflicts; not general NLI.

# 7 Related Work

mem0 [1] provides overwrite-style hosted memory. Zep [2] offers temporal knowledge graphs. Engram [3] implements bi-temporal storage. Supersede [6] identifies the "memory update gap" but does not prove impossibility. TRACE [7] formalizes temporal memory admission. MemStrata [8] implements bi-temporal ledgers. ConflictBank [9] provides a large-scale conflict benchmark. Our work combines an impossibility result, a causal experiment on real production data, and a practical resolution system—no prior work combines all three.

# 8 Conclusion

We proved that temporal correctness in agent memory requires storage-layer support. We showed that the temporal gap causes 39–73pp downstream accuracy loss. We implemented and evaluated a bi-temporal resolution system achieving 100% temporal accuracy with flat $O(\log n)$ latency. Code and data: `https://github.com/lanbass869-cell/cac-artifact`.

# References


[1] P. Chhikara et al. Mem0: Building Production-Ready AI Agents. arXiv:2504.19413, 2025.

[2] R. Rasmussen et al. Zep: A Temporal Knowledge Graph. arXiv:2501.13956, 2025.

[3] Engram: Bi-Temporal Memory with Hybrid Retrieval. arXiv:2606.09900, 2026.

[4] LangChain. LangMem: Memory for LangGraph. Software, 2025.

[5] MemTether: Local-First Governed Memory. arXiv preprint, 2026.

[6] Supersede: Memory Update Gap in LLM Agents. arXiv:2606.27472, 2026.

[7] TRACE: Temporal Memory Admission Framework. arXiv:2609.33517, 2026.

[8] N. Yadav. MemStrata: Bi-Temporal Ledger. arXiv:2606.26511, 2026.

[9] ConflictBank: A Benchmark for Knowledge Conflicts. arXiv, 2024.

[10] StateMemBench: Evaluating State Tracking. arXiv, 2026.

[11] TWIST: Measuring Intervention Quality. arXiv, 2026.

[12] C. Wu et al. LongMemEval. arXiv:2410.10813, 2024.